AIMERZ.ai
Aim.Act.Achieve

NOTES

Java Object-Oriented Programming (OOP) - II

Instance Variables and Local Variables

Static Vs Instance Members

Understanding Instance & Local Variables in Java

What are Variables?
Think of variables as containers that store information in our program. Just like we store things in different types of boxes at home, we store different types of data in variables.

Types of Variables in Java
In Java, we mainly deal with two important types of variables:

1. Instance Variables
2. Local Variables

Let's understand with Real-World Example:
Think of a restaurant:

Instance Variables are like:
- Restaurant's name
- Restaurant's address
- Restaurant's rating

These stay with the restaurant all the time

Local Variables are like:
- Today's special dish
- Number of current customers
- Daily earnings

These are temporary and change daily

What are Instance Variables?
Instance Variables (Object Variables)

- An instance variable is a piece of information about an object.
- It's like a characteristic or property that belongs to the object itself.
- For example, in a class `Car`, an instance variable `colour` would hold the car's colour.

Rules for Instance Variables:
Declaration:
- Must be inside class
- Must be outside methods
- Can use access modifiers (public, private, protected)

Memory for Instance Variables:
- Get default values if not initialized
- int → 0
- double → 0.0
- boolean → false
- object references → null

Lifetime of Instance Variables:
-Instance variables live as long as the object exists
- Start: When object is created using 'new'
- End: When object is destroyed by garbage collector

Real-World Example 1: Student Management

```Java
public class Student {
    // Instance Variables
    String name;        // Each student has their own name
    int rollNumber;     // Each student has their own roll number
    double marks;       // Each student has their own marks
    boolean isPassed;   // Each student has their own pass status

    // Constructor
    public Student(String studentName, int studentRoll) {
        name = studentName;
        rollNumber = studentRoll;
        marks = 0.0;          // Starting marks
        isPassed = false;     // Starting pass status
    }
}
```

Think of it like a school:
Each student (object) has their own:
* ID Card (rollNumber)
* Name tag (name)

* Report card (marks)
* Pass certificate (isPassed)

What are Local Variables ?

What are Local Variables?

- A local variable is a temporary piece of information used inside a method.
- It's like a "helper" that exists only while the method is running.
- For example, inside a `drive()` method, a local variable `speed` would store the speed of the car while driving.

Rules for Local Variables:

Declaration:
- Must be inside a method
- Cannot use access modifiers
- Must be initialized before use

Memory:

- No default values
- Must give value before using
- Created in method's stack frame

Lifetime Local Variables:

-Local variables only exist while their method is running
- Start: When method starts
- End: When method ends

Real-World Example: Shopping Calculator

```Java
public class ShoppingCalculator {
    double taxRate = 0.10;  // Instance variable - same for all calculations

    public double calculateTotal(double price) {
        // Local Variables
        double discount = 5.0;           // Local discount amount
        double discountedPrice = price - discount;  // After discount
        double tax = discountedPrice * taxRate;     // Calculate tax
```

```
        double finalPrice = discountedPrice + tax;  // Final price

        return finalPrice;
    }
}
```

Key Differences

Scope (Where we can use them):

```Java
public class Phone {
    String brand = "Samsung";
    // Instance variable - available everywhere in  class

    public void makeCall() {
        int duration = 5;
        // Local variable - only in makeCall method
        System.out.println(brand + " phone calling for " + duration + "
minutes");
    }

    public void sendMessage() {
        System.out.println(brand + " phone sending message");
        // Cannot use duration here - it's a local variable of makeCall()
    }
}
```
```
```

Lifetime (How long they exist):

```java
public class Game {
    int score = 0;    // Instance variable - exists as long as game object exists

    public void playLevel() {
        int levelBonus = 100;  // Local variable - exists only during level play
        score = score + levelBonus;
    }
    // levelBonus no longer exists here
}
```

Common Mistakes to Avoid

Not Initializing Local Variables:

```java
public void wrongExample() {
    int number;              // Local variable
    System.out.println(number);  // ERROR! Must initialize before use
}

public void correctExample() {
    int number = 0;         // Initialized local variable
    System.out.println(number);  // Works fine
}
```

Trying to Access Local Variables Outside Their Method:

```java
public class Example {
    public void firstMethod() {
        int localVar = 10;    // Local variable
    }

    public void secondMethod() {
```

```
        // System.out.println(localVar);  // ERROR! Cannot access localVar
here
    }
}
```

Practice Exercise com Mini Project

Try creating a simple program about a library book:
- Instance variables should store:
  * Book title
  * Author name
  * Number of pages
  * Whether it's borrowed

- Local variables in methods should handle:
  * Days borrowed
  * Late fees
  * Current page being read

A Note:
- Instance Variables:
  * Belong to objects
  * Live throughout object's life
  * Have default values
  * Can be used anywhere in class

- Local Variables:
  * Belong to methods
  * Live only during method execution
  * Must be initialized
  * Can only be used in their method

Project Implementation

```
Java
public class LibraryBook {
    // Instance variables (belong to the LibraryBook object)
```

```java
    private String title;
    private String author;
    private int numberOfPages;
    private boolean isBorrowed;

    // Constructor to initialize the LibraryBook instance
    public LibraryBook(String title, String author, int numberOfPages) {
        this.title = title;
        this.author = author;
        this.numberOfPages = numberOfPages;
        this.isBorrowed = false; // Default to not borrowed
    }

    // Method to borrow the book
    public void borrowBook(int daysBorrowed) {
        if (!isBorrowed) {
            isBorrowed = true;
            System.out.println("The book has been borrowed for " +
daysBorrowed + " days.");
            // Local variable for calculating late fees
            double lateFee = calculateLateFee(daysBorrowed);
            System.out.println("Estimated late fee if returned late: $" +
lateFee);
        } else {
            System.out.println("The book is already borrowed.");
        }
    }

    // Method to return the book
    public void returnBook() {
        if (isBorrowed) {
            isBorrowed = false;
            System.out.println("The book has been returned.");
        } else {
            System.out.println("The book wasn't borrowed.");
        }
    }

    // Method to read a certain number of pages
    public void read(int currentPage, int pagesToRead) {
        if (currentPage + pagesToRead <= numberOfPages) {
            System.out.println("You read " + pagesToRead + " pages from page
" + currentPage + ".");
        } else {
            System.out.println("You've reached the end of the book.");
        }
    }
```

```
    // Private method to calculate a late fee (not accessible outside this
class)
    private double calculateLateFee(int daysBorrowed) {
        // Local variable to hold the fee per day (arbitrary value for
example)
        double feePerDay = 0.50;
        // Calculate total fee based on days borrowed
        return daysBorrowed * feePerDay;
    }

    // Method to display book details
    public void displayBookDetails() {
        System.out.println("Title: " + title);
        System.out.println("Author: " + author);
        System.out.println("Pages: " + numberOfPages);
        System.out.println("Is borrowed: " + isBorrowed);
    }

    // Main method to test the LibraryBook class
    public static void main(String[] args) {
        // Create a new LibraryBook object
        LibraryBook book = new LibraryBook("The Great Peace", "Swamy Vieveka
Nanda", 180);

        // Display book details
        book.displayBookDetails();

        // Borrow the book for 7 days
        book.borrowBook(7);

        // Try reading 50 pages from page 20
        book.read(20, 50);

        // Return the book
        book.returnBook();

        // Display book details after returning
        book.displayBookDetails();
    }
}

Output:

Title: The Great Peace
Author: Swamy Vieveka Nanda
Pages: 180
Is borrowed: false
```

```
The book has been borrowed for 7 days.
Estimated late fee if returned late: $3.5
You read 50 pages from page 20.
The book has been returned.
Title: The Great Peace
Author: Swamy Vieveka Nanda
Pages: 180
Is borrowed: false
```

Static vs. Instance Members in Java

What Are They?

- **Static Members**: These are variables and methods that belong to the class itself, not to any specific object created from the class. This means there is only one copy of a static member, and it is shared by all instances of the class.
- **Instance Members**: These are variables and methods that belong to a specific object (or instance) of the class. Each object has its own copy of instance members, and these can have different values for different objects.

Real world Analogy: Library

Think of a library to understand the difference:

- **Static Members**: These are like the general rules and resources of the library (such as opening hours, membership fees, and the list of available books). These rules are the same for every visitor to the library.
- **Instance Members**: These are like the individual library cards and personal details of each visitor (such as their name, borrowed books, and due dates). These details are unique to each visitor.

Main Points:

Shared vs. Unique:

- Static members (like library rules) are shared among all visitors (instances). Everyone follows the same rules.

- Instance members (like library cards) are unique to each visitor. Each person has their own card with personal information.

Accessing Members:

- You access static members using the class name (like looking up the library's rules).
- You access instance members using an object of the class (like showing your library card to access your account).

Memory Allocation:

- Static members are allocated memory once when the class is loaded (like having one set of library rules).
- Instance members are allocated memory each time a new object is created (like each visitor having their own library card).

Example in Java:

```java
class Library {
    // Static member (shared)
    static String libraryName = "City Library";

    // Instance members (unique)
    String visitorName;
    int visitorID;

    // Constructor
    Library(String name, int id) {
        this.visitorName = name;
        this.visitorID = id;
    }

    // Static method
    static void displayLibraryInfo() {
        System.out.println("Welcome to " + libraryName);
    }

    // Instance method
    void displayVisitorInfo() {
        System.out.println("Visitor Name: " + visitorName + ", Visitor ID: "
+ visitorID);
    }
}

public class StaticVsInstanceExample {
    public static void main(String[] args) {
        // Accessing static method and member
        Library.displayLibraryInfo();

        // Creating instances (visitors)
        Library visitor1 = new Library("Ajay", 101);
        Library visitor2 = new Library("Vijay", 102);
```

```
        // Accessing instance methods and members
        visitor1.displayVisitorInfo(); // Outputs: Visitor Name: Alice,
Visitor ID: 101
        visitor2.displayVisitorInfo(); // Outputs: Visitor Name: Bob,
Visitor ID: 102

        // Static member remains the same across all instances
        System.out.println("Library Name (from visitor 1): " +
visitor1.libraryName);
        System.out.println("Library Name (from visitor 2): " +
visitor2.libraryName);
    }
}
```

Output

```
Java
Welcome to City Library
Visitor Name: Ajay, Visitor ID: 101
Visitor Name: Vijay, Visitor ID: 102
Library Name (from visitor 1): City Library
Library Name (from visitor 2): City Library
```

Code Key Features:

Static Member: libraryName is a static variable shared among all instances of Library, representing a common attribute of the library.

Instance Members: visitorName and visitorID are instance variables unique to each Library object, allowing different visitors to have their own details.

Static Method: The method displayLibraryInfo() can be called without creating an instance of Library, demonstrating how static methods can operate on class-level data.

Final Conclusion:

Static members are like the general rules and resources of a library that apply to all visitors, while instance members are like the personal library cards that contain unique information for each visitor. Understanding this

distinction helps in designing classes and managing resources effectively in Java.

Key Differences Between Static Variables & Instance Variables

Feature	Instance Variables	Static Variables
Ownership	Unique to each object.	Shared across all objects of the class.
Memory Allocation	Allocated when an object is created.	Allocated once when the class is loaded.
Modification	Changes affect only the specific object.	Changes affect all objects.
Access	Accessed via objects.	Accessed via class name (no object required).
Life Cycle	Exists as long as the object exists.	Exists until the program terminates or the class is unloaded.

THANK YOU

AIMERZ.ai

Aim.Act.Achieve

Stay updated with us!

Vishwa Mohan

Vishwa Mohan

vishwa.mohan.singh

Vishwa Mohan